

Relazione del Progetto di Programmazione per la Fisica

Simulazione dinamica ed emergenza collettiva del comportamento di stormi nel modello dei Boids con SFML

Borchetta Simone, Genovesi Antonio

28 agosto 2026

Modifiche dalla consegna precedente

Rispetto alla consegna del 15 luglio 2026 il progetto è stato sottoposto a modifiche e revisioni. Riportiamo di seguito i cambiamenti salienti:

- **Unificazione del namespace:** Nella stesura precedente, per evitare conflitti di nomi, il codice era stato ripartito in diversi namespace indipendenti (`phy`, `boids`, `stats`, `sim`). Per rispettare rigorosamente le specifiche del progetto, tutte le entità del programma (tipi, funzioni, classi) sono state ora migrate e dichiarate all'interno di un unico namespace (`bs`) dedicato alla simulazione
- **Strumento di Formattazione:** Come richiesto dalle linee guida, il file di configurazione `.clang-format` utilizzato per definire le regole stilistiche del codice è stato esplicitamente incluso nell'archivio di consegna, insieme al file `CMakeLists.txt` e alle altre dipendenze.
- **Ottimizzazione delle dichiarazioni e delle definizioni delle Funzioni (Parametri e Return Types):** È stata condotta una revisione sistematica delle interfacce delle funzioni. Si è prestata particolare attenzione al passaggio appropriato dei parametri, implementando il passaggio per const reference (`const T&`) per gli oggetti complessi (es. le strutture dati della configurazione o vettori di grandi dimensioni) e mantenendo il passaggio per valore (by value) per i tipi primitivi. Parallelamente, sono stati ottimizzati i return types per prevenire copie non necessarie.
- **Uso di `this` e rimozione di `self_index`, cambiamento dei return type di `Boid::update`:** Sono stati implementati dei range-for loop nelle funzioni di aggiornamento della simulazione. Dopo aver tentato di far riconoscere al boid se stesso mediante un approccio geometrico non sempre efficace (controllando che la distanza tra i due boid fosse pari a 0), abbiamo deciso di modificare i metodi della classe `Boid` ad esempio: `Boid::compute_cohesion`, `Boid::compute_alignment`, `Boid::compute_separation...` in modo da riconoscere il boid attraverso gli indirizzi di memoria. A tal proposito abbiamo dovuto modificare interamente la logica di programmazione, dal momento che inizialmente si confrontavano `boids` appartenenti a due flock diversi (`flock`, `next_flock`). Come conseguenza è variato il codice inerente al metodo `Boid::update` che restituisce una copia modificata che viene, in seguito all'applicazione di forze attraverso `Boid::apply_force`, inserita in `next_flock`.
- **Uso appropriato di `static`:** Abbiamo eliminato l'utilizzo di `static` nel codice.
- **Uso di `std::sqrt`:** Rispetto alla precente versione del progetto è stato limitato l'utilizzo della funzione `std::sqrt` della Standard Library. Abbiamo convertito diversi controlli limitandoli a confronti tra quadrati delle variabili. In questo modo è stato realizzato un codice più leggero e ottimizzato rispetto alle versioni precedenti.

- **Uso di `std::generate_n`:** È stato implementato l'algoritmo di generazione `std::generate_n` all'interno di `std::vector<Boid> entity_gen`, al fine di evitare il precedente ciclo `for`, quindi rendere più espressivo e ottimizzato il codice.
- **Modifiche relative alla relazione:** Dati i cambiamenti del codice abbiamo provveduto a perfezionare la descrizione di quest'ultimo nella relazione da consegnare. Abbiamo inoltre aggiunto nuove spiegazioni in quest'ultima relative alle configurazioni iniziali della simulazione. È stata aggiunta una sotto sezione riguardante la spiegazione degli algoritmi non spiegati a lezione frontale e comunque implementati nel codice. Infine sono stati documentati a fondo relazione i risultati ottenuti mediante l'utilizzo di immagini.

Riassunto

Il progetto implementa una simulazione numerica del modello dei *Boids*, introdotto da Craig Reynolds nel 1986 per descrivere il comportamento collettivo e coordinato (*flocking*) di stormi di uccelli o banchi di pesci. Ogni agente evolve nel tempo seguendo semplici regole locali geometriche e cinematiche. La simulazione è interamente realizzata in C++ ed integra la libreria grafica SFML per la visualizzazione bidimensionale in tempo reale, una componente statistica per l'analisi dei dati in esecuzione e un modulo interattivo che prevede la presenza di agenti predatori (*hunters*) e forze esterne pilotabili dall'utente tramite mouse.

1 Introduzione e Tema del Progetto

Il presente lavoro implementa uno dei temi d'esame del corso. L'obiettivo principale è studiare simulazione delle proprietà emergenti di un sistema complesso auto-organizzato, basato sul modello di *flocking* dei Boids.

All'inizio della simulazione, le posizioni e le velocità dei boids sono del tutto casuali e disordinate. Tuttavia, le interazioni a corto raggio tra i vicini fanno in modo che si formi uno stormo unito e coerente. Il funzionamento del modello si basa sul concetto di *proprietà emergente*: non esiste un leader che coordina il movimento, ma ogni agente decide come muoversi guardando solo gli altri boids che si trovano all'interno del suo raggio visivo (d). Ad ogni passo temporale Δt , il ciclo di esecuzione prevede che ogni boid scansioni l'ambiente circostante, identifichi i propri vicini e calcoli vettorialmente le forze d'interazione. La somma di questi contributi determina la nuova velocità e, di conseguenza, la posizione successiva dell'agente sullo schermo.

2 Modello Fisico e Regole di Volo

Le regole di volo servono a calcolare le variazioni di velocità da applicare a ciascun boid. Matematicamente, l'aggiornamento della velocità $\vec{v}_{b_i}$ e della posizione $\vec{x}_{b_i}$ di ogni boid b_i avviene tramite le seguenti equazioni:

$$\vec{v}_{b_i} = \vec{v}_{b_i} + \vec{v}_1 + \vec{v}_2 + \vec{v}_3 \quad (1)$$

$$\vec{x}_{b_i} = \vec{x}_{b_i} + \vec{v}_{b_i} \Delta t \quad (2)$$

I vettori $\vec{v}_1$, $\vec{v}_2$ e $\vec{v}_3$ sono i contributi calcolati applicando le tre regole di Reynolds sui boids vicini. Dato il raggio visivo d , l'insieme dei vicini N_i del boid i -esimo contiene tutti gli agenti che si trovano a una distanza minore di d :

$$N_i = \{b_j \mid 0 < \|\vec{x}_i - \vec{x}_j\| < d\} \quad (3)$$

2.1 Le Regole Classiche di Reynolds

Il comportamento complessivo dello stormo emerge dalla combinazione lineare di tre forze fondamentali:

- **Separazione:** Ha lo scopo di evitare che i boids collidano tra loro. L'accelerazione repulsiva si attiva solo se la distanza mutua è inferiore a un raggio critico di sicurezza $d_s < d$:

$$\vec{v}_1 = -s \sum_{j \neq i} (\vec{x}_{b_j} - \vec{x}_{b_i}) \quad \text{se } \|\vec{x}_{b_i} - \vec{x}_{b_j}\| < d_s \quad (4)$$

L'uso della distanza come fattore cinematico garantisce un puntamento e un allontanamento graduale, la cui intensità è regolata dal coefficiente s .

- **Allineamento:** Fa in modo che gli agenti tendano a procedere nella stessa direzione dello stormo, sincronizzando i vettori di velocità relativi:

$$\vec{v}_2 = a \left(\frac{1}{n-1} \sum_{j \neq i} \vec{v}_{b_j} - \vec{v}_{b_i} \right) \quad (5)$$

Il contributo si ottiene sottraendo la velocità attuale del boid b_i dalla media delle velocità dei suoi vicini, pesata per un fattore $a < 1$ che determina la rapidità di sterzata.

- **Coesione:** Induce l'agente a volare verso il baricentro locale dei boids circostanti per mantenere l'unità dello stormo. Il centro di massa $\vec{x}_c$ è calcolato come:

$$\vec{x}_c = \frac{1}{n-1} \sum_{j \neq i} \vec{x}_{b_j} \quad (6)$$

La componente di velocità $\vec{v}_3$ si ricava di conseguenza come attrazione verso tale baricentro tramite un coefficiente di accoppiamento c :

$$\vec{v}_3 = c(\vec{x}_c - \vec{x}_{b_i}) \quad (7)$$

2.2 Modellizzazione della Predazione: gli *Hunters*

Per inserire un elemento di disturbo nello stormo, il programma prevede la presenza dei cacciatori (*hunters*). Rispetto ai boids ordinari, gli *hunters* hanno parametri diversi: possiedono dimensioni geometriche maggiori e una velocità massima più elevata.

Il loro diverso comportamento segue dall'applicazione di forze centrali attraverso la funzione `apply_force`:

- **Separazione tra i Cacciatori:** Gli *hunters* non si muovono in modo del tutto isolato; per evitare di andarsi a sbattere addosso a vicenda, la regola della separazione rimane attiva all'interno del loro gruppo.
- **Propulsione:** Ciascun cacciatore viene spinto costantemente nella direzione in cui si sta già muovendo. Il programma calcola una forza diretta verso un punto proiettato in avanti nel tempo lungo la sua traiettoria ($\vec{x}_{\text{hunter}} + 4\vec{v}_{\text{hunter}}$) al fine di rendere più fluida la simulazione.
- **Attrazione verso il Centro dello Stormo:** Contemporaneamente, il cacciatore viene attratto dal centro di massa dello stormo dei boids ($\vec{x}_c$). Questa forza centrale serve a fargli inseguire il gruppo, e la sua intensità viene dimezzata (moltiplicata per -0.5) rispetto al parametro standard.

Non appena un *hunter* entra nel raggio di percezione dei boid (`influence_radius`), i boids vengono spinti da una forza repulsiva che si somma vettorialmente alle altre forze dello stormo ma, data la sua intensità, domina la deviazione della traiettoria spingendoli nella direzione opposta.

3 Scelte Progettuali e Ripartizione del Codice

Il codice è scritto in C++ su diversi file:

- **vector2d.hpp**: Contiene le definizioni dell'interfaccia per la struttura dati **Vector2D**, progettata per la gestione di vettori nello spazio bidimensionale $\mathbb{R}^2$.
- **vector2d.cpp**: Contiene le definizioni delle funzioni della struttura dati dichiarata nell'header **vector2d.hpp**. Inoltre sono presenti gli overload degli operatori dichiarati in esso.
- **boid.hpp**: Definisce la classe **Boid**, che costituisce l'agente fondamentale della simulazione. Questa classe incapsula lo stato fisico dell'entità (posizione e velocità) e i metodi necessari per implementare il comportamento collettivo basato sul modello di Reynolds (separazione, allineamento e coesione).
- **boid.cpp**: Implementa la logica comportamentale e cinetica della classe **Boid**. Basandosi sul modello di Reynolds, il codice traduce le interazioni locali tra le entità in vettori di forza, determinando il movimento fluido e collettivo tipico di uno stormo.
- **statistics.hpp**: Contiene le dichiarazioni dell'interfaccia per il modulo di analisi statistica, progettato per elaborare i dati relativi a un gruppo di oggetti di tipo **Boid** (tipicamente rappresentanti una simulazione di stormo).
- **statistics.cpp**: Fornisce l'implementazione logica delle funzioni di analisi statistica per lo stormo di **Boid**. Utilizza algoritmi della libreria standard C++ (`<algorithm>`, `<numeric>`) per elaborare in modo efficiente le proprietà del vettore *flock*.
- **simconfig.hpp**: Definisce la struttura dati **SimConfig**, che centralizza tutti i parametri di configurazione necessari al motore di simulazione. Tale approccio permette di separare la logica dei parametri dai componenti dinamici (i *Boid*), facilitando la gestione delle impostazioni sia globali che specifiche per eventuali sottogruppi (es. predatori o *kettle*). Abbiamo implementato questo *header file*, incluso ad esempio in **boid.cpp** e in **boid.hpp**, in modo da poter dichiarare e definire delle funzioni che accettassero in input una singola struct, al posto di svariati parametri. Sono state dichiarate delle funzioni necessarie alla configurazione della simulazione, qualora si volessero utilizzare dei parametri da passare attraverso `std::cin`.
- **simconfig.cpp**: Implementa la logica di inizializzazione e configurazione dinamica dei parametri di simulazione. Gestisce l'interazione con l'utente tramite standard input per definire le caratteristiche dello stormo e dei predatori (*hunter*), includendo meccanismi di validazione dei dati in ingresso.
- **simulation.hpp**: Contiene le dichiarazioni dell'interfaccia per il modulo di aggiornamento fisico della simulazione. In esso è presente la dichiarazione della funzione `update_physics`, che gestisce l'evoluzione temporale dello stormo (*flock*) e di eventuali sotto-gruppi (*kettle*), applicando le regole di comportamento (*flocking*) basate sulle configurazioni (**SimConfig**) e sugli input dell'utente (posizione del mouse e stati dei tasti). Inoltre, è presente la funzione `entity_gen` per la generazione dello stormo, meglio descritta di seguito.
- **simulation.cpp**: Implementa la logica di aggiornamento temporale della simulazione (`update_physics`). Esso funge da motore principale, coordinando il calcolo delle forze, l'applicazione delle regole di comportamento (*flocking*) e la gestione dei vincoli spaziali per le entità (*Boid*) presenti sia nello stormo principale che nel sotto-gruppo dei predatori (*kettle*).
- **render.hpp**: Viene dichiarata l'interfaccia per il modulo di visualizzazione grafica, che utilizza la libreria SFML (*Simple and Fast Multimedia Library*) per l'interfacciamento con l'hardware grafico. Il modulo si occupa della traduzione dello stato logico della simulazione (vettori di entità e dati statistici) in rappresentazioni visive su finestra.

- **render.cpp:** Contiene l'implementazione del motore grafico del progetto, che traduce lo stato numerico della simulazione (posizioni, velocità e dati statistici) in un output visivo mediante la libreria *SFML*. Il modulo segue un pattern di rendering immediato, ottimizzato per l'aggiornamento costante durante il ciclo di gioco (*game loop*).

Nell'archivio inviato sono presenti file come `CMakeLists.txt`, `.clang-format`, `doctest.h`, `arial.ttf` utilizzati nello sviluppo del progetto.

3.1 Scelte Algoritmiche e Dinamica delle Forze

3.1.1 Configurazione Iniziale e Parametri di Input

Per garantire flessibilità nell'analisi della simulazione, la funzione `get_user_config()` richiede all'avvio alcuni parametri di input da terminale standard. L'utente può scegliere tra due modalità principali di inizializzazione:

- **Configurazione Predefinita:** Avvia immediatamente la simulazione caricando un set di parametri fisici standard (velocità massime, coefficienti delle tre regole e raggi di influenza) pre-impostati per garantire uno stormo stabile e visivamente coerente.
- **Configurazione Personalizzata:** Consente all'utente di modificare manualmente i singoli coefficienti per studiare comportamenti alternativi e fluttuazioni del sistema.

Per agevolare le procedure di configurazione della simulazione ed evitare l'inserimento manuale interattivo a ogni avvio, l'acquisizione dei parametri tramite `std::cin` è stata progettata per supportare la redirectione dell'input da un file di testo. È sufficiente creare un file (ad esempio quello da noi consegnato nell'archivio `configuration.txt`) contenente i valori disposti sequenzialmente, separati da ritorni a capo, nello stesso ordine in cui vengono richiesti dal prompt del programma.

A titolo di esempio, il contenuto del file per una configurazione personalizzata con 600 boids e 2 predatori si presenta in questo modo:

Esempio di `configuration.txt` commentato

```
n      % Impostazione configurazione personalizzata
      % (Riga vuota)
600    % Numero di boids
60     % Visual range (d)
20     % Separation radius (ds)
0.8    % Separation factor (s)
0.5    % Alignment factor (a)
0.01   % Cohesion factor (c)
      % (Riga vuota)
2      % Numero di hunters
```

Indipendentemente dalla modalità scelta, il sistema richiede esplicitamente l'inserimento del **numero di cacciatori (*hunters*)**, anche nullo, da generare.

Esempio di esecuzione con configurazione personalizzata Di seguito si riporta un esempio di input interattivo consigliato per la valutazione del comportamento collettivo:

```
Would you like to start the boid simulation with the default parameters? (
y/n): n
Enter number of boids: 150
Enter the visual range (d): 60
Enter the separation radius (ds): 20
Enter the separation factor (s): 0.8
Enter the alignment factor (a): 0.5
```

```
Enter the cohesion factor (c): 0.01
Enter number of hunters: 3
```

3.1.2 Generazione di flocks e kettles

La generazione sia degli stormi di Boid (*flocks*) che dei gruppi di predatori (*kettle*) è affidata alla funzione `entity_gen`. La funzione riceve in input un oggetto di tipo `bs::SimConfig` contenente tutti i parametri necessari per configurare l'entità. La funzione esegue un controllo sugli input: per gli stormi di *boids* viene richiesto un numero di entità strettamente positivo, mentre per gli *hunters* un numero non negativo. In caso di parametri non validi, il sistema solleva un'eccezione `std::runtime_error`. Utilizzando il motore di numeri casuali `std::default_random_engine` la funzione distribuisce le entità nello spazio $\mathbb{R}^2$. Le coordinate (x, y) sono calcolate entro i bordi definiti dalla configurazione, applicando un margine di sicurezza. Ogni entità viene inizializzata con un vettore velocità (v_x, v_y) casuale e controlla il valore booleano `is_hunter` presente nella configurazione: se esso ha valore `true`, vengono applicati i limiti di velocità specifici del predatore.

3.1.3 Regole di volo

Sono state implementate le regole di volo presentate in Sezione 2:

- `compute_separation`: impedisce ai boids di sovrapporsi o scontrarsi. Per ogni vicino all'interno del `separation_radius` calcola un vettore repulsivo che allontana il boid corrente. L'intensità della spinta è regolata attraverso il `separation_factor`.
- `compute_alignment`: permette allo stormo di muoversi in modo coordinato. Il boid calcola la media dei vettori velocità dei vicini entro il proprio `visual_range` e genera una forza che tende ad allineare la propria direzione a quella del gruppo, pesata dall'`alignment_factor`.
- `compute_cohesion`: spinge il boid verso il centro di massa dei vicini percepiti, garantendo che lo stormo rimanga coeso e non si disperda, regolata dal `cohesion_factor`.

Sono state costruite delle funzioni di supporto al movimento delle entities:

- `limit_velocity`: assicura che ogni boid rimanga entro un intervallo di velocità realistico (`min_vel` e `max_vel`), evitando accelerazioni infinite che renderebbero instabile la simulazione.
- `apply_toroidal_boundary`: crea uno spazio ciclico (se un boid esce a destra, rientra a sinistra).
- `apply_window_boundary`: genera una forza repulsiva verso l'interno quando il boid si avvicina ai bordi della finestra.
- `apply_force`: permette di applicare influenze dirette, come l'attrazione/repulsione causata dal mouse o dai predatori, calcolando un vettore basato sulla distanza rispetto a un punto di influenza.

3.1.4 Evoluzione temporale

L'evoluzione temporale del sistema adotta un approccio di aggiornamento sincrono delle coordinate. All'interno della funzione `update_physics`, lo stato dello stormo e del kettle al tempo t viene letto per calcolare i contributi di forza su ciascun agente; le nuove posizioni e velocità al tempo $t + \Delta t$ vengono quindi generate e salvate nei vettori ausiliari `next_flock` e `next_kettle`. Successivamente, il riassetto dello stato avviene tramite `std::move(next_flock)` e `std::move(next_kettle)`. Questo approccio elimina asimmetrie temporali fittizie dovute all'ordine sequenziale dell'elaborazione nei cicli `for`.

L'evoluzione dinamica ad ogni passo di integrazione si articola nelle seguenti fasi:

1. **Aggiornamento dei Boids:** Il metodo `update` della classe `Boid` calcola le componenti di accelerazione dovute alle tre regole di Reynolds (separazione, allineamento e coesione) calcolate rispetto agli altri componenti dello stormo. A queste si aggiunge la forza repulsiva generata da ciascun *hunter* presente nel vettore `kettle`, simulando l'istinto di fuga dei boids. Vengono infine applicati i vincoli di bordo (toroidali o confinati) per mantenere le entità nell'area di simulazione.
2. **Aggiornamento dei Hunters** Il codice gestisce una flotta di *hunters* distinta (`kettle`), caratterizzata da limiti di velocità superiori. I cacciatori inseguono il centro di massa dei boids percepiti e mantengono una distanza di sicurezza tra loro tramite la sola regola di volo attiva: quella di separazione. Le forze integrate nel loro vettore velocità ne determinano lo spostamento finale.
3. **Interattività Utente:** L'utente può perturbare il sistema in tempo reale tramite input asincroni del mouse. La pressione del tasto destro genera una forza attrattiva verso le coordinate del cursore, mentre il tasto sinistro applica una forza repulsiva.

3.1.5 Generazione dei grafici attraverso SFML

Il modulo di rendering grafico è stato sviluppato, come anticipato, nel file `render.cpp`, che si basa sull'aggiornamento ciclico di primitive grafiche all'interno di una `sf::RenderWindow`. Il file si occupa di tre funzionalità principali:

1. Rendering delle entità: le funzioni `render_flock` e `render_kettle` visualizzano gli stormi e i predatori attraverso oggetti di tipo `sf::ConvexShape`. Le forme vengono dichiarate come `static` con un flag `initialized`, che garantisce che la geometria venga definita una volta sola all'avvio, risparmiando risorse computazionali durante il *game loop*. Per ogni entità il codice esegue due operazioni prima di richiamare `window.draw()`: vengono allineate la forma alle coordinate (x, y) del boid attraverso la funzione `set_Position`. Attraverso la funzione `set_Rotation` viene orientato il triangolo in base all'angolo del vettore velocità, permettendo all'utente di percepire la direzione del moto del boid.
2. Rappresentazione dei dati: la funzione `render_histogram` trasforma un insieme di dati statistici (bins di velocità) in un istogramma grafico. L'altezza di ogni barra viene calcolata proporzionalmente al valore massimo presente nel dataset (`max_count`) rispetto all'altezza totale disponibile (`bounds.height`). Il ciclo `for` crea rettangoli (`sf::RectangleShape`) per ogni bin, posizionandoli orizzontalmente in modo equispaziato. Vengono utilizzate `sf::Text` per visualizzare i valori numerici e le etichette dell'asse X. Il font viene caricato una sola volta staticamente, evitando operazioni di I/O disco onerose a ogni fotogramma.
3. Gestione degli Eventi: la funzione `handle_events` funge da interfaccia di controllo tra l'utente e la simulazione. Si utilizza `window.pollEvent` per monitorare costantemente le azioni dell'utente, come la chiusura delle finestre. La pressione di tasti specifici modifica lo stato della simulazione in tempo reale: premendo il tasto 'B' si cambia la modalità dei confini (toroidal $\leftrightarrow$ bounded), premendo il tasto 'S' apre o chiude la finestra secondaria dedicata alle statistiche, gestendo dinamicamente il ciclo di vita della `statsWindow` (creazione e distruzione).

3.2 Strategia di Test e Verifica del Codice

Per garantire l'affidabilità del simulatore e la correttezza delle equazioni fisiche implementate, si è adottato il framework di unit testing `doctest`. La suite automatizzata è suddivisa in tre macro-aree che validano le componenti critiche del sistema:

- **Algebra vettoriale (Vector2D):** I test verificano la rigorosa correttezza aritmetica degli overload degli operatori (+, -, *, /, ecc.). Oltre al calcolo della norma euclidea e della distan-

za, viene validata la funzione `angle()` rispetto ai quattro punti cardinali. È stata inoltre testata la robustezza del codice contro operazioni matematicamente non definite, accertandosi che la divisione per zero (es. `v / 0.0`) sollevi correttamente un'eccezione `std::runtime_error`.

- **Logica e cinematica degli agenti (Boid):**

- *Vincoli cinematici:* Verifica del corretto troncamento del modulo della velocità quando questo supera `max_vel` o scende sotto `min_vel`.
- *Regole di Reynolds:* Calcolo manuale incrociato: i test istanziano stormi di dimensioni ridotte (2 o 3 boids) con coordinate note, verificando che i vettori restituiti da `compute_separation`, `compute_alignment` e `compute_cohesion` coincidano esattamente (tramite `doctest::Approx`) con i risultati teorici. Sono stati testati anche i casi limite (stormi vuoti o composti da un solo boid, e raggio visivo nullo) per escludere la generazione di valori NaN.
- *Forze e Confini:* Validazione delle condizioni al contorno: i test assicurano il corretto riposizionamento delle entità in modalità toroidale (wrap-around) e il calcolo esatto della forza repulsiva elastica ai margini della finestra. Infine, viene verificato il corretto funzionamento di `apply_force` nel generare vettori di fuga/inseguimento (es. rispetto alla posizione di un *hunter*).

- **Integrazione fisica e Generazione (simulation):**

- *Ciclo di simulazione:* Verificata l'evoluzione temporale Δt (`update_physics`), delle interazioni dinamiche con il mouse (attrazione e repulsione), della reazione di fuga dai predatori e del corretto comportamento dei confini.
- *Generazione entità:* Controllo del posizionamento casuale degli agenti all'intero della finestra (`entity_gen`) e gestione delle eccezioni (`std::runtime_error`) con parametri di popolazione negativi o nulli ($N \leq 0$).

- **Modulo Statistico (stats):** Si verifica la correttezza del calcolo della media e della deviazione standard per posizione e velocità. Oltre ai campioni standard, il sistema è testato per gestire stormi vuoti restituendo vettori nulli, evitando così crash a runtime. Per quanto riguarda l'istogramma, i test convalidano la corretta allocazione delle entità nei *bin* (inclusa la convergenza delle velocità fuori scala nell'ultimo bin disponibile) e la corretta gestione delle eccezioni in caso di input non validi (es. numero di bin nullo o velocità massima negativa).

3.3 Costrutti non trattati a lezione

Nello sviluppo del codice relativo al progetto sono stati implementati degli algoritmi non trattati a lezione. Di seguito, in tabella (1) riportiamo i principali costrutti utilizzati.

Tabella 1: Algoritmi non trattati a lezione

Algoritmo	caratteristiche
<code>std::move</code>	Nella funzione <code>update_physics</code> , l'aggiornamento dello stato dello sciame segue uno schema di double buffering. Per evitare dipendenze dall'ordine di iterazione (in cui il calcolo delle forze per il boid <i>i</i> -esimo verrebbe influenzato dalle posizioni già aggiornate dei boid precedenti), i nuovi stati vengono accumulati in due vettori locali temporanei, <code>next_flock</code> e <code>next_kettle</code> . Al termine del ciclo, occorre aggiornare i vettori di stato principali (<code>flock</code> e <code>kettle</code>). Invece di eseguire una copia elemento per elemento, si ricorre a <code>std::move</code> . Viene creato un oggetto nuovo ri utilizzando le risorse di proprietà di un altro.

<code>std::transform_reduce</code>	È stato preferito l'utilizzo dell'algoritmo <code>std::transform_reduce</code> della Standard Library al fine di utilizzare del codice ben implementato e ottimizzato. L'algoritmo combina in un'unica iterazione la mappatura di ogni elemento (fase di transform) e l'accumulazione dei risultati (fase di reduce).
<code>std::cin.clear</code> e <code>std::cin.ignore</code>	Se il flusso <code>std::cin</code> entra in uno stato di errore, tutte le successive operazioni di lettura falliscono silenziosamente. La funzione <code>clear_cin_buffer()</code> definita in <code>simconfig.cpp</code> è costruita (attraverso l'utilizzo di <code>std::cin.clear</code> e <code>std::cin.ignore</code>) per ripristinare lo stato dello stream e svuotare completamente il buffer residuo, garantendo che le successive richieste di input funzionino correttamente e senza blocchi.
<code>std::plus</code>	Nell'algoritmo <code>std::transform_reduce</code> , serve a specificare l'operazione di riduzione, ossia come combinare tra loro i valori estratti da ciascun Boid.
<code>std::vector::reserve</code>	L'istruzione <code>std::vector::reserve(n)</code> alloca preventivamente la memoria dinamica nello <i>heap</i> per contenere almeno n elementi, modificando la capacità del vettore senza alterarne la dimensione (<code>size()</code> , che rimane 0).

4 Guida all'Uso e Parametri di Input

4.1 Dipendenze e Librerie Esterne

Il progetto richiede il supporto della libreria grafica multimediale SFML (Simple and Fast Multimedia Library). Il programma è sviluppato per ambienti Linux (incluso WSL per Windows). I pacchetti necessari e la libreria SFML si installano da terminale digitando:

```
sudo apt update
sudo apt install libsFML-dev
```

4.2 Istruzioni per la Compilazione e il Testing

Per la gestione della compilazione viene utilizzato CMake in combinazione con Ninja Multi-Config. Questa scelta permette di configurare la cartella di *build* una sola volta e di compilare successivamente il codice sia in modalità Debug (per l'esecuzione dei test) sia in modalità Release (ottimizzata per la simulazione), senza dover rigenerare l'ambiente da capo.

La configurazione iniziale si esegue posizionandosi nella cartella radice del progetto e digitando:

```
cmake -S . -B build -G "Ninja Multi-Config"
```

Per compilare il programma in modalità *Debug* ed eseguire i test unitari automatizzati, i comandi sono:

```
cmake --build build --config Debug
cmake --build build --config Debug --target test
```

Infine, per generare la versione definitiva e ottimizzata in modalità *Release* ed effettuare il controllo finale tramite i test, si esegue:

```
cmake --build build --config Release
cmake --build build --config Release --target test
```

Per eseguire la simulazione ci si posiziona all'interno della directory che include la cartella `build` e si esegue il comando:

```
$ ./build/Debug/simulazione
```

per avviare la simulazione in modalità Debug, oppure si avvia in modalità Release:

```
$ ./build/Release/simulazione
```

5 Formato di Output e Risultati

5.1 Formato dei Canali di Output

La simulazione si biforca in tre canali di output simultanei:

1. **Visualizzazione Grafica Principale:** Una finestra SFML renderizza i boids sotto forma di triangoli rossi orientati lungo il proprio vettore velocità e i cacciatori come triangoli di colore azzurro.
2. **Finestra Statistica Ausiliaria:** Premendo il tasto `S` sulla tastiera, viene istanziata una seconda finestra grafica indipendente che disegna l'istogramma dinamico delle frequenze di velocità del sistema, discretizzato in 25 bin calibrati su una velocità massima di 50.0.
3. **Output di Diagnostica da Terminale:** Lo standard output (`std::cout`) stampa ogni 60 cicli le grandezze fisiche medie e le deviazioni standard dello stormo nel seguente formato:

```
Velocita' media: 27.42 | Dev.Std: 3.12 | Posizione media: 580.11 |
Dev.Std: 142.34
Velocita' media: 27.51 | Dev.Std: 2.98 | Posizione media: 581.45 |
Dev.Std: 139.88
```

Premendo il tasto `B` sulla tastiera è possibile alternare dinamicamente tra confini chiusi (finestra finita) o condizioni toroidali (periodiche).

5.2 Interpretazione Fisica dei Risultati

Le rilevazioni statistiche evidenziano come, partendo da distribuzioni casuali omogenee prodotte dal generatore pseudorandomico `std::uniform_real_distribution`, il sistema evolva verso uno stato stazionario ordinato.

In regime toroidale e in assenza di *hunters*, la deviazione standard del modulo della velocità diminuisce rapidamente, indicando che gli agenti hanno accoppiato con successo i propri vettori di volo (dunque coordinano i loro movimenti), riducendo le velocità relative (fase di *flock* compatto). L'inserimento di agenti predatori rompe questo equilibrio, inducendo la dispersione locale dei boids limitrofi e la formazione di sottogruppi coordinati di fuga, riscontrabili nelle improvvise fluttuazioni della deviazione standard spaziale stampata a terminale.

Gestione del Progetto (GitHub): Per la scrittura del codice il team ha adottato `Git` come sistema di controllo versione e `GitHub` come repository remoto. Questa infrastruttura ci ha permesso di isolare lo sviluppo di nuove feature su *branch* separati, garantendo un'integrazione sicura del codice tramite *merge* e risolvendo i conflitti di sovrascrittura. Di seguito riportiamo l'indirizzo della repository: <https://github.com/Micron14/phy-prog>.

5.3 Documentazione dei risultati

Di seguito, in figura (1), riportiamo attraverso l'utilizzo di *screenshots* i risultati ottenuti attraverso l'esecuzione del nostro codice:

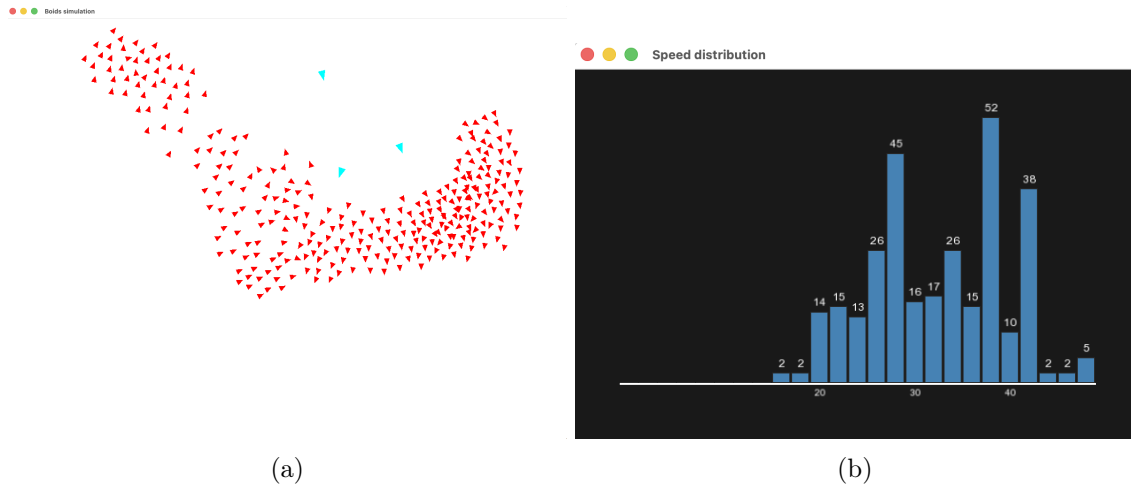


Figura 1: Simulazione di uno stormo mediante il modello di Reynolds: in figura (a) in rosso sono rappresentati i **boids** mentre in azzurro sono disegnati gli **hunters**, in (b) la finestra mostra la distribuzione delle velocità dei **boids**.

6 Dichiarazione sull'uso di IA Generativa

In accordo con i vincoli di trasparenza del corso, si dichiara che il modello di Intelligenza Artificiale Generativa Gemini è stato utilizzato esclusivamente come supporto per lo studio delle funzioni della libreria SFML (la quale esulava dal programma approfondito a lezione), per la ricerca di algoritmi e l'interpretazione di messaggi di errore del compilatore. La progettazione logica finale del sistema, l'implementazione software delle equazioni fisiche in C++, l'integrazione delle chiamate grafiche e il debugging finale rimangono interamente frutto dell'analisi e del lavoro originale del gruppo.